

Текст Т3

Данные: <https://budget.gov.ru/epbs/registry/ubpandnubp/data>

Апи: <https://budget.gov.ru/%D0%9E%D1%82%D0%BA%D1%80%D1%8B%D1%82%D1%8B%D0%B5-%D0%B4%D0%B0%D0%BD%D0%BD%D1%8B%D0%B5?code=7710568760-RUBPNUBP>

они отдаются страницами, параметр page

Основное задание:

1. скачать данные из источника (все страницы за указанный период)
2. разложить данные в таблицу (дополнительных таблиц не создавать, должна получиться одна денормализованная таблица). Предусмотреть в программе случай, когда пользователь решит перезагрузить данные на даты, ранее уже загруженные (перезаписываем их по ключам lastUpdateFrom и lastUpdateTo)
3. Ранее скачанные данные из пункта 1 собираем в zip-архив

В качестве входных параметров программы: lastUpdateFrom и lastUpdateTo

По итогу должно получиться консольное приложение, которое выполняет описанные выше функции. Основное требование расположения данных в таблице, это не помещать полностью тело запроса в одно поле. Массивы в таблицу записывать целиком нельзя как json/jsonb», каждый тег в json - атрибут в денормализованной таблице.

Стек: java 8, maven(для java), postgresql 11+

Можно использовать библиотеки для java:

1. jackson для json
2. для http запросов http-components
3. драйвер postgresql
4. junit для тестов

Готовое задание выложить на Github

Дополнительно:

Будет плюсом, если вы добавите, какие-то фишки, перечисленные ниже:

1. Сохранять в zip-архив xml-файлы, из ранее загруженных json-файлов
2. Дополнительно организовать шаг переноса данных из postgresql в clickhouse
3. Использовать в своей работе Docker, Docker Compose для разворачивания экземпляров БД

Решение

Аннотация

Этот файл является заменой README.md, т.к. тут я собираюсь изложить все свои мысли возникшее в процессе выполнения работы, для чего README.md не очень-то подходит. Сначала расскажу о том, как я интерпретировал текст задания – были некоторые неоднозначные моменты, затем напишу о самом приложении – как\что\где работает и т.д.

Файл написан по большой степени в свободной форме, но в любом случае **рекомендован к прочтению**.

Интерпретация ТЗ

«Скачать данные из источника (все страницы за указанный период)» - программа должна уметь делать запросы и как-то обрабатывать их.

«Предусмотреть в программе случай, когда пользователь решит перезагрузить данные на даты, ранее уже загруженные (перезаписываем их по ключам lastUpdateFrom и lastUpdateTo)» - Когда мы вставляем в БД сущность, которая уже находится там (для проверки уникальности используем guid) мы перезаписываем всю строку, заменяя все значения, в том числе и параметры запроса (lastUpdateFrom и lastUpdateTo).

«каждый тег в json - атрибут в денормализованной таблице» - [см. Денормализация данных](#).

«Дополнительно организовать шаг переноса данных из postgresql в clickhouse» - именно перенос данных, а не загрузка в оба места после получения данных с API.

Таже обращаю внимание на то, что нас просят использовать Java 8 и другой, фиксированный набор технологий. Это требование учтено, использованы только технологии из списка. Однако, для удобства разработки и отладки было принято решение добавить Lombok и библиотеку логирования в зависимости проекта.

Анализ данных

Рассмотрим пример данных, с которыми мы имеем дело. Перейдём по ссылке (<https://budget.gov.ru/epbs/registry/ubpandnubp/data>) и увидим структуру стандартного ответа.

```
{
  "offset": -1,
  "pageNum": 1,
  "pageSize": 10,
  "recordCount": 351836,
  "data": [ 10 elements ],
  "order": "id",
  "orderDirection": "asc",
  "limits": {
  },
  "version": "10",
  "pageCount": 35184
},
{
  "contacts": [
    {
      "phone": "8 346 645 00 91; 8 346 646 20 43",
      "site": "http://mbdoy34.ucoz.ru",
      "mail": "nv-madoy34@mail.ru"
    }
  ],
  "acceptAuths": [],
  "transfauth": [],
  "data": [
    { 20 keys },
    { 20 keys },
    { 20 keys },
    { 20 keys },
    { 20 keys },
    { 20 keys },
    { 20 keys },
    { 20 keys },
    { 20 keys },
    { 20 keys },
    {
      "id": "3320358",
      "info": {
        "regNum": "00308",
        "code": "74300308",
        "divisionParentName": "",
        "divisionParentCode": "",
        "ogrn": "1028600966536",

```

Мы видим, что ответ содержит какое-то количество сущностей (data: [...]), которые содержат большой набор различных атрибутов разных уровней вложенности. Имеется блок info, поля которого имеют одно значение, и поля других блоков (например contacts), которые представлены наборами других сущностей.

На сайте есть ссылка на «документацию» по приходящим данным.

10	Описание структуры набора открытых данных	7710568760-RUBPNUBP.docx	7710568760-RUBPNUBP.json
----	---	--	--

Это файл, с описанием всех полей, их типов и т.д. Отметим, **что реальные данные расходятся с их описанием**. Так, например в ответе у нас есть поле attachment, однако оно никак не упоминается в документации. Присутствуют и других поля, содержание/описание которых не соответствует действительности. Учтем это в дальнейших шагах.

Денормализация данных

Изучив структуру ответа, мы можем перейти к шагу проектировки структуры БД и таблицы. В требованиях указано, что нам разрешено использовать только 1 таблицу, но при этом – цитирую: «каждый тег в json - атрибут в денормализованной таблице».

Для удобства рассмотрим структуру, похожую на ответ, но не такую большую.

```
{
  "id": 1,
  "name": "Krymov Valerii",
  "alias": [
    {
      "id": 1,
      "name": "Crumbobly"
    },
    {
      "id": 2,
      "name": "vkvalera004@gmail.com"
    },
    ...inf...
  ]
}
```

У нас также есть поля хранящие 1 значение, и поля, содержащие не фиксированный (потенциально бесконечный) набор других сущностей.

Как хранить такие объекты в БД:

1. Первое идей было хранить только ключи первого уровня:
“id”: 1, “name”: “Krymov Valerii”, “alias”: “Сырой json”.
Однако это нарушает требования к табличке БД. Мы и храним json (пусть и как TEXT) и про «каждый тэг – атрибут» можно поспорить.
2. Второй идей было хранить вообще все данные в формате «ключ: значение», не допуская хранения множества значений. Эта идея была частично реализована посредством динамического изменения количества столбцов (**ветка dynamic**), однако мы столкнулись с проблемой огромного количества столбцов, которое Postgres не может создать (ограничение на ~1600 столбцов)
Набор столбцов для примера: id, name, alias_1_id, alias_1_name, alias_2_id, alias_2_name, ..., alias_n_id, alias_n_name.

Очевидно, что хранить все атрибуты как «конечные значения» не представляется возможным. По крайней мере не с текущим стеком технологий.
3. Затем было принято наиболее оптимальное, на мой взгляд, решение:
Мы будем хранить все атрибуты сущности следующим образом:
“id”: 1, “name”: “Valerii Krymov”, “alias_id”: “1#2#...”, “alias_name”:
“Crumbobly#vkvalera004@gmail.com#...”.
- выступает в роли разделителя.

Так у нас будет относительно небольшое количество столбцов, при этом мы будем хранить данные в полном объеме. В случае, если часть данных отсутствует (=null), например name у alias с id=2 равен null мы получим:

“id”: 1, “name”: “Valerii Krymov”, “alias_id”: “1#2#...”, “alias_name”: “Crumbobly##...”.

Таблицы БД

Логика создания таблиц и на Postgres и на Clickhouse примерно одинакова:

Все ключи первого уровня (в примере – id=1, name=“Krymov Valerii” мы будем хранить в тех типах, что описаны в документации. Остальные поля будут храниться в текстовом представлении, т.к. будут (могут) содержать набор значений. Для примера:

Id – int, **name** – String

Предположим есть **date_birth** – Date

alias_id – String (т.к. оно представляет собой набор id – «1#2#...»)

alias_name – String

Предположим есть **alias_date_create** – String

В процессе работы были сложности с корректным определением типов данных. Например, в документации написано, что поле имеет тип Bool, но по факту это не так. Также были проблемы с сопоставлением типов Postgres – Clickhouse. Date в Postgres может хранить большой диапазон дат, а Date (или Date32, или Datetime64) в Clickhouse хранит меньший, недостаточный диапазон. Это проблема возникла несколько раз, но для повышения безопасности в Clickhouse мы будем использовать String для хранения дат.

Из-за того что мы не смогли корректно определить является ли поле обязательным – большинство полей были помечены как null-содержащие.

Docker + БД

Разворачивать экземпляры БД будем в Docker. Для настройки (user, password, db_name) у нас есть переменные окружения (.env). Также вся инициализация (создание индексов, таблиц, других объектов) и другие запросы (select, insert) происходит посредством выполнения заранее заготовленных скриптов в папке **resources/db/***.

Архитектура приложения

Приложение построено по принципу последовательного конвейера (пайплайна) и состоит из нескольких этапов:

1. Постраничное получение данных из API (budget.gov.ru).
2. Преобразование JSON в DTO-объекты.
3. Сохранение сырых JSON-файлов на диск.
4. Формирование XML-файлов из DTO-объектов.
5. Денормализация (flatting) структуры данных.
6. Загрузка данных в PostgreSQL.
7. Архивация JSON/XML файлов в ZIP.
8. Перенос данных из PostgreSQL в ClickHouse.

Каждый этап выделен в отдельный сервис (java-класс).

Синхронизация PostgreSQL и ClickHouse

Перенос выполняется отдельно от загрузки данных из API. Синхронизация выполняется следующим образом:

1. После загрузки всех данных определяется момент начала синхронизации (для простоты взята дата-время старта программы).
2. Из PostgreSQL выбираются записи, измененные после указанного времени.
3. Данные считываются кусками, что позволяет не загружать всю таблицу в память.
4. И вставляются в ClickHouse.

Процесс синхронизации можно (в теории, в коде такой реализации нет) вызывать отдельно. Не обязательно предварительно обращаться к API и загружать данные вообще.

Возможные улучшения

Возможные улучшения:

- использовать бинарный-протокол ClickHouse вместо JDBC;
- увеличить покрытие тестов;
- использовать Flyway или другие фреймворки для управления БД;
- при увеличении кодовой базы – использовать Spring, или другие DI решения.
- Уделить внимание структуре данных (неточности документации и пр.) и прийти к единому «стандарту» хранения.
- Провести дополнительную оптимизацию использования оперативной памяти, пересмотрев структуру хранения и жизненный цикл объектов в приложении.

Производительность

Аппаратная конфигурация:

- ОС: Windows 11 Pro (10.0.26200)
- Процессор: Intel Core i5-10500H
- Оперативная память: 24 ГБ DDR4 (16 ГБ + 8 ГБ, 3200 МГц)

Сетевое подключение:

- Тип подключения: Wi-Fi
- Скорость подключения: 100 Мбит/с
- Провайдер: (необязательно)

Тестовая среда:

- PostgreSQL 17 (Docker)
- ClickHouse 26.5.1 (Docker)
- Java 8
- Maven

Был выполнен запрос данных за период 01.01.2020 – 31.12.2100, получено 351823 записей.

Суммарное время выполнения этапов:

Этап	Время
Получение данных из API	~60 мин

Этап	Время
Формирование архива	~4 мин
Синхронизация PostgreSQL -> ClickHouse	~5 мин
Общее время выполнения	~70 мин

Основным узким местом является скорость работы внешнего API budget.gov.ru.

Вторым узким местом является скорость синхронизации Postgres и Clickhouse. Перенос данных выполняется со скоростью порядка 1000 записей в секунду. Логи этого запуска можно найти в файле `log.txt`.

Инструкция по запуску

Перед запуском необходимо создать файл `(.env)` в корневой директории проекта и указать параметры подключения к базам данных. Пример необходимых переменных можно найти на [github](https://github.com).

Из корневой директории проекта выполнить команду: `(docker compose up -d)`.

Будут подняты контейнеры:

- PostgreSQL;
- ClickHouse.

Проект собирается с помощью Maven: `(mvn clean package)`. После сборки будет создан обычный JAR-файл. Fat JAR содержащий все зависимости, не формировался, поэтому запуск командой `(java -jar _____.jar)` не поддерживается.

Запуск приложения рекомендуется выполнять непосредственно из IDE (IntelliJ IDEA) с использованием класса `Application`, предварительно убедившись, что:

- файл `(.env)` содержит необходимые переменные окружения;
- контейнеры PostgreSQL и ClickHouse запущены.

При необходимости проект может быть доработан для формирования исполняемого Fat JAR содержащего все зависимости.